PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

Smart Music Shuffler: Fair, Repeat-Free Playlist Shuffling with Custom Data Structures in C++

Team Name: *Team #*	**DSCPP-III-2026-T142**
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	Singhal, Bhuvi – 2510010271 2510010271@geu.ac.in 
Team member 2 *(Last Name, name: student ID: email, picture):*	Joshi, Akshat – 2510012740 2510012740@geu.ac.in 
Team member 3 *(Last Name, name: student ID: email, picture):*	Chauhan, Akshank – 2510010897 2510010897@geu.ac.in 
Team member 4 *(Last Name, name: student ID: email, picture):*	Singh, Tansi – 2510010500 2510010500@geu.ac.in 

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

(Brief restatement of your project's main goal. Max 300 words).

Smart Music Shuffler is a console-based playlist manager and shuffler written in C++, in which every data structure is implemented from scratch without the Standard Template Library (STL). Simple random playback often repeats songs before the whole playlist has been heard, and naive shuffling methods produce biased orderings in which some sequences are more likely than others. The project addresses both problems. Songs are loaded from a text file into a circular doubly linked list that supports insertion, deletion by ID and traversal in both directions. The playlist is shuffled with the Fisher–Yates algorithm, which runs in $O(n)$ time and makes every ordering equally likely; this fairness is verified statistically rather than assumed. A custom open-addressing hash set records which songs have already played so that no song repeats within a playback cycle, and a stack-based history lets the user return to previously played songs. The final application will play local `.wav` files through the Windows multimedia API. The goal is a working terminal audio player whose correctness and fairness are shown through measurable tests, while giving the team hands-on experience with linked lists, hashing, stacks, algorithm analysis and manual memory management in C++.

Updated Project Approach and Architecture (2 pts)

(Describe your current approach, including system design, communication protocols, libraries used, etc. Max 300 words).

The system is a standalone console application with a modular design: each component is a separate C++ module (header and source file), and all modules share a common `Song` structure (`id`, `title`, `artist`).

Data flow. `playlist.txt` stores one song per line in the form `id|title|artist`. **FileParser** (`loadPlaylist`) reads it, tolerating blank lines and missing fields, and fills the **Playlist**, a circular doubly linked list. To shuffle, the list is copied into an array (`toArray`), shuffled in place by **FisherYates** (`shuffleSongs`), and written back (`fromArray`), because Fisher–Yates needs random access that a linked list lacks.

Tracking. CustomHashSet stores IDs of played songs using modulo hashing and open addressing with linear probing; it doubles its capacity and rehashes when load exceeds 70%, and implements the Rule of Three for safe copying. **HistoryStack** is an array-based stack (default capacity 200) of previously played songs.

Planned integration (Phase-III). A playback controller with a menu will connect these modules: next, previous, reshuffle, and clearing the hash set and reshuffling when every song has played. Audio will use the Win32 `PlaySound` function from `mmsystem.h` with the `SND_ASYNC` flag, so playback runs in the background without freezing the terminal. This needs a file-path field in `Song`.

Libraries and tools. `iostream`, `fstream`, `cstring`, `cstdlib`, and `mmsystem.h` for audio; no STL containers or algorithms, no database, and no network communication. Targets Windows; compiled with GCC or MSVC; managed with Git and GitHub.

Tasks Completed (7 pts)

(Describe the main tasks that have been assigned and already completed. Max 250 words).

Task Completed	Team Member
Playlist: circular doubly linked list with add, delete by ID (including head), display and array conversion	Bhuvi Singhal
Fisher–Yates shuffle with permutation and position fairness tests	Akshat Joshi
CustomHashSet: open addressing, linear probing, auto-rehash at 70% load, deep copy	Akshank Chauhan
HistoryStack: push, pop and peek of play history (undo side)	Tansi Singh

Challenges/Roadblocks (7 pts)

(Describe the challenges that you have faced or are facing so far and how you plan to solve them. Max 300 words).

No STL. Strings, arrays and memory are handled manually with C-strings and `new/delete`, which risks leaks and double frees. We added destructors to every class and the Rule of Three to the hash set, and ran the tests under AddressSanitizer, which found no errors or leaks. `Playlist` and `HistoryStack` still lack copy constructors, so copying them is unsafe; we will add these or disable copying.

Circular list traversal. A circular list has no `NULL` end, so loops are bounded by the song count to avoid infinite traversal.

Shuffling a linked list. Fisher–Yates needs random access, so the list is copied to an array, shuffled and written back, at an $O(n)$ space cost.

Proving fairness. A few outputs cannot show a shuffle is unbiased, so we ran chi-square tests over thousands of shuffles.

Randomness. `rand()` is not yet seeded, so every run produces the same order. The playback controller will call `srand` once at start-up.

Integration. The modules work individually but are not yet connected; this is the main work for Phase-III.

Tasks Pending (7 pts)

(Describe the main tasks that you still need to complete. Max 250 words).

Task Pending	Team Member (to complete the task)
Playback controller (next, previous, reshuffle; reset the hash set when a cycle ends); seed the random generator; add copy safety to Playlist and HistoryStack	Bhuvi Singhal
Menu-driven console interface (`main.cpp`); README and final documentation	Akshat Joshi
Win32 audio playback of `.wav` files (`PlaySound`, `SND_ASYNC`); add file path to `Song`	Akshank Chauhan
Redo stack to complete undo/redo navigation; integration testing and memory test log for the complete application	Tansi Singh

Project Outcome/Deliverables (2 pts)

(Describe what are the key outcomes / deliverables of the project. Max 200 words).

1. A working terminal audio player in C++ that plays local `.wav` files in non-repeating shuffled order, built without the STL.
2. Five reusable data-structure modules: circular doubly linked list, Fisher–Yates shuffle, open-addressing hash set, history stack and file parser.
3. Measured results: statistically verified unbiased shuffling, no repeated song within a playback cycle, and working undo/redo navigation.
4. The compiled `.exe`, `playlist.txt`, test results, and a memory test log showing destructors free all memory.
5. A GitHub repository with individual commits from every member, a README, and the final report and presentation.

Progress Overview (2 pts)

(Summarize how much of the project is done, what's behind schedule, what's ahead of schedule. Max 200 words.)

The Phase-I proposal set three milestones: (1) architecture planning, (2) C++ implementation of the data structures, and (3) integration with Win32 audio plus memory profiling.

On schedule: Milestone 1 is complete. Milestone 2 is nearly complete: FileParser, Playlist, Fisher–Yates shuffle and CustomHashSet (including manual collision probing) are implemented and pass all tests. HistoryStack supports undo; redo is not yet built.

Ahead of schedule: part of Milestone 3's memory profiling is already done. The module tests run under AddressSanitizer with no leaks, and shuffle fairness has been verified with chi-square tests.

Remaining for Phase-III: the playback controller and menu, Win32 `.wav` playback, integration testing and the final memory test log. Estimated completion: **about 60%**.

Codebase Information (2 pts)

(Repository link, branch, and information about important commits.)

Repository: `https://github.com/rixezz/Music-Shuffler` **Branch:** `main`

Structure:

```
Music-Shuffler/
├── .gitignore
├── CustomHashSet.cpp
├── CustomHashSet.h
├── FisherYates.cpp
├── FisherYates.h
├── Playlist.cpp
├── Playlist.h
└── Report/
    └── Phase 1/
        ├── Docs/
        │   └── Phase-1.docx
        ├── Pdf/
        │   └── Phase-1.pdf
        └── ppt.pdf
```

Commits: 8 commits on `main` as of 9 October 2026, including the Phase-I report and the Fisher–Yates and Playlist modules. Each member commits their own module from their own GitHub account: FileParser and Fisher–Yates (Akshat Joshi), Playlist (Bhuvi Singhal), CustomHashSet (Akshank Chauhan) and HistoryStack (Tansi Singh).

Testing and Validation Status (2 pts)

(Provide information about any tests conducted)

Test Type	**Status (Pass/Fail)**	**Notes**
Unit: FileParser	Pass	100 of 100 songs loaded from `playlist.txt`
Unit: Playlist (circular DLL)	Pass	Links consistent both ways; head and middle deletion; missing ID reported
Unit: shuffle round-trip	Pass	All songs present exactly once after shuffling
Statistical: 3-element permutations (60,000 runs)	Pass	Each ordering 16.55–16.81% (ideal 16.67%); $\chi^2 = 2.09 < 11.07$
Statistical: 100-song positions (10,000 runs)	Pass	Song 0 in each slot 0.78–1.25% (ideal 1%); $\chi^2 = 91.14 < 123.23$
Unit: CustomHashSet	Pass	Duplicates and negative IDs rejected; collisions resolved; capacity 32→256; deep copy correct
Unit: HistoryStack	Pass	LIFO order; overflow and underflow handled
Memory (AddressSanitizer)	Pass	No memory errors or leaks during the test run
Integration (full application)	Pending	Playback controller not yet built

Deliverables Progress (2 pts)

(Summarize the current status of all key project deliverables mentioned earlier. Indicate whether each deliverable is completed, in progress, or pending.)

1. Data-structure modules: **Completed** for FileParser, Playlist, Fisher–Yates and CustomHashSet; **In progress** for HistoryStack (redo pending).
2. Integrated terminal audio player and `.exe`: **Pending**.
3. Testing: **Completed** for module tests; integration tests and memory test log **Pending**.
4. GitHub repository: **In progress**, with members committing their modules.
5. README, final report and presentation: **In progress**.